

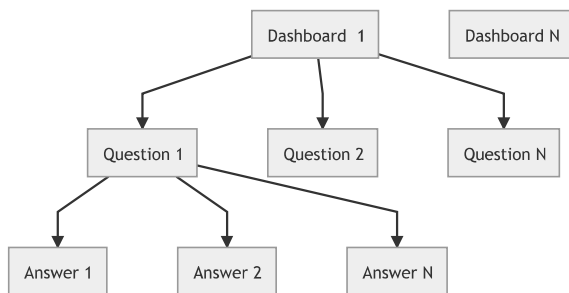
Mostrando una lista de respuestas (I y II)

Resumen

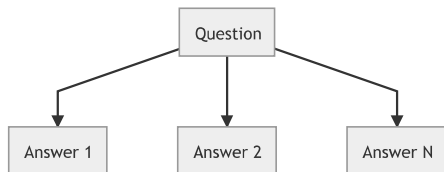
- Crearemos un componente `AnswerListItem.js` que será usado para pintar una celda que muestra una *respuesta*
- Crearemos `QuestionDetailPage.js` y lo enlazaremos con una `<Route>` a <http://localhost:3000/dashboards/:idDashboard/questions/:idQuestion>
- En `QuestionDetailPage.js` pediremos el *detalle de una pregunta* (lista de respuestas) y las almacenaremos en un estado `answers`, además de otros datos
- Mediante `answers.map` transformaremos cada elemento en un `<AnswerListItem>`

Descripción

Recordemos que en nuestro proyecto hay tres niveles de jerarquía lógicos:



Ahora es momento de trabajar en el último eslabón:



Así que, si lo deseas, puedes eliminar el comentario que dejamos hace un par de tareas en `QuestionListItem.js`:

```
const onClick = () => {  
  - // Esto navega a una URL que todavía no hemos implementado...  
  - // TO-DO: Implementar detalle de pregunta (lista de respuestas)  
  - // Lo haremos en un par de tareas  
  navigate('questions/'+props.question.question_id);  
}
```

La tarea

Crema `src/components/answer_list_item/AnswerListItem.css` con el siguiente CSS:

```
.answer-li {  
  margin: 29px;  
  padding: 12px;  
  background-color: rgb(238, 200, 200);  
  text-align: center;  
  box-shadow: rgba(0, 0, 0, 0.15) 1.95px 1.95px 2.6px 0px;  
}
```

Luego, **crea** un nuevo componente: `src/components/answer_list_item/AnswerListItem.js` con **estas** características:

- **Tendrá** un `<div className='answer-li' key={props.answer.answer_id}>` raíz
- Dicho `<div>` **contendrá** un `<p>{props.answer.description}</p>`

A continuación, **crea** `src/screens/question/QuestionDetailPage.js`:

```
import { useParams } from "react-router-dom";
```

```
const QuestionDetailPage = (props) => {
  const params = useParams();

  return <div></div>
}

export default QuestionDetailPage;
```

...y en **App.js** **incluye** la siguiente ruta:

```
<Route path="/dashboards/:idDashboard/questions/:idQuestion" element={<QuestionDetailPage/>}></Route>
```

Ya puedes verificar lo que vayas haciendo desde:

- <http://localhost:3000/dashboards/1/questions/1>

A continuación, en **QuestionDetailPage.js** **añade** los siguientes estados:

```
const [questionTitle, setQuestionTitle] = useState('');
const [questionDescription, setQuestionDescription] = useState('');
const [answers, setAnswers] = useState([]);
```

Y **añade** este **useEffect** para pedir al API REST los datos detallados de la *pregunta*:

```
useEffect(() => {
  axios.get('http://raspi:8081/api/v1/dashboards/' + params.idDashboard + '/questions/' + params.idQuestion).then(response => {
    setQuestionTitle(response.data.question_title);
    setQuestionDescription(response.data.question);
    setAnswers(response.data.answers);
  })
}, [])
```

Y, para terminar, **modifica** el **return** en **QuestionDetailPage.js** para que sea el siguiente:

```
return <div data-cy='issue23body'>
  <h1 data-cy='issue23title'>{questionTitle}</h1>
  <p data-cy='issue23description'>{questionDescription}</p>
  <div data-cy='answersList'>
    { answers.map((a) => { return <AnswerListItem answer={a}/> }) }
  </div>
</div>
```

Verifica tu resultado.



Por último

Sube tus cambios al repositorio en un nuevo *commit*.



[Rubén Montero @ruben.montero](#) changed milestone to [%Sprint 3](#) 1 week ago